Projeto do Robô

Entrega 1 — Rota de menor custo com recursão

Grupo 06

Prof. Jemerson Damásio

Linguagem: Python

Integrantes

Leonardo Lucas Braz Araújo

Carlos Adrians Araújo Damacena

Matheus Pinto Clemente

Sumário

- 1. Descrição do problema
- 2. Solução recursiva
- 3. Testes
- 4. Por que funciona
- 5. Medição do trabalho
- 6. Regra gulosa
- 7. Conferência com os critérios de avaliação
- Apêndice — Como executar e arquivos entregues

1. Descrição do problema

Um robô está no **canto superior esquerdo** de uma grade retangular e precisa chegar ao **canto inferior direito**. Cada casa tem um custo de energia. O objetivo é encontrar o **menor custo total** possível para fazer esse trajeto, ou informar que o trajeto é impossível.

Regras

- O robô anda **uma casa por vez**, e só para a **direita** ou para **baixo**.
- Ele **não sai da grade** e **não entra em obstáculos** (casas marcadas com X, representadas por `null` no JSON).
- O custo de uma rota é a **soma de todas as casas visitadas**, incluindo a casa de partida e a de chegada.
- Se a partida ou a chegada forem obstáculos, ou se os obstáculos bloquearem todas as rotas, não existe caminho.

Entrada

A entrada é um **texto** no formato JSON com a chave `custos`, que contém a grade como uma lista de linhas. Exemplo (a grade do enunciado):

```
{"custos": [[2, 1, 9, 9, 9],
            [3, null, 1, 1, 9],
            [1, 1, 1, null, 2],
            [9, 9, 1, 1, 1]]}
```

A entrada é válida somente quando:

- o texto é um JSON bem formado e é um objeto com a chave `custos` (outras chaves são ignoradas);
- `custos` é uma **l**ista não vazia de linhas;
- cada linha é uma lista não vazia e todas as linhas têm o mesmo tamanho;
- cada valor é um inteiro positivo (1, 2, 3, ...) ou `null`. Não valem `true`/`false`, textos como `"1"`, números com casas decimais como `1.5`, zero ou negativos.

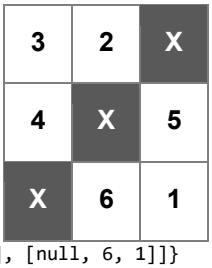
Saída

A função `resolver(entrada)` devolve um dicionário Python com um destes formatos:

Situação	Saída
Existe rota	`{"status": "ok", "custo": 11}`
Entrada válida, sem rota	`{"status": "sem_caminho", "custo": null}`
Entrada inválida	`{"status": "entrada_invalida", "erro": "mensagem"}`

Exemplos próprios

a) Entrada válida sem rota. Todos os valores são inteiros positivos ou null e as linhas têm o mesmo tamanho, então a entrada é válida. Porém, da partida (3) o robô só pode ir para o 2 (direita) ou para o 4 (baixo). Do 2, a direita e a casa de baixo são obstáculos; do 4, a casa de baixo e a da direita também são. O robô fica cercado e nunca chega ao 1 da chegada.




```
Entrada: {"custos": [[3, 2, null], [4, null, 5], [null, 6, 1]]}
Saída:   {"status": "sem_caminho", "custo": null}
```

b) Entrada inválida. O valor é um texto e não uma lista de linhas.

```
Entrada: {"custos": "1 2 3"}
Saída:   {"status": "entrada_invalida", "erro": "'custos' deve ser uma lista não vazia de linhas"}
```

2. Solução recursiva

A recursividade funciona fazendo a função chamar a si mesma para encontrar o menor custo até a chegada. Em cada casa, ela verifica os caminhos possíveis, sempre para a direita ou para baixo, e escolhe aquele que possui o menor custo. Se encontrar um obstáculo ou não existir caminho até o destino, retorna None. O contador registra quantas vezes a função foi chamada. Como não existe memorização, os resultados não são guardados, fazendo com que algumas casas sejam calculadas mais de uma vez.

Validação e função de contrato

resolver(entrada) lê o texto com o leitor de JSON da linguagem, confere as regras de validade com validar_custos e só então chama recursivo.

3. Testes

Os testes ficam em tests/test_robo.py, usam o módulo unittest (já vem com o Python) e rodam sozinhos com o comando python -m unittest discover -s tests -v. Cada caso compara a saída de resolver com o resultado esperado; nos casos de entrada inválida o teste confere o status e se há uma mensagem em erro.

3.1 Casos exigidos pelo professor

Entrada	Esperado	Obtido	
[[7]]	ok, custo 7	ok, custo 7	OK
[[1, 2, 3]]	ok, custo 6	ok, custo 6	OK
[[1], [2], [3]]	ok, custo 6	ok, custo 6	OK
[[1, 1, 1], [1, 1, 1]]	ok, custo 4	ok, custo 4	OK
Grade do exemplo	ok, custo 11	ok, custo 11	OK
[[1, 1, null], [2, null, 1], [1, 1, 1]]	ok, custo 6	ok, custo 6	OK
[[null]]	sem_caminho	sem_caminho	OK
[[null, 1], [1, 1]]	sem_caminho	sem_caminho	OK
[[1, 1], [1, null]]	sem_caminho	sem_caminho	OK
[[1, null], [null, 1]]	sem_caminho	sem_caminho	OK
custos = []	entrada_invalida	entrada_invalida	OK
custos = [[]]	entrada_invalida	entrada_invalida	OK
custos = [[1, 2], [3]]	entrada_invalida	entrada_invalida	OK
custos = [[true, 1]]	entrada_invalida	entrada_invalida	OK
custos = [["1", 1]]	entrada_invalida	entrada_invalida	OK
custos = [[1.5, 1]]	entrada_invalida	entrada_invalida	OK
custos = [[0, 1]]	entrada_invalida	entrada_invalida	OK

custos = [[-1, 1]]	entrada_invalida	entrada_invalida	**OK**
Texto {"matriz": [[1]]}	entrada_invalida	entrada_invalida	**OK**
Texto [[1, 2]]	entrada_invalida	entrada_invalida	**OK**
Texto {"custos": [[1, 2]	entrada_invalida	entrada_invalida	**OK**

3.2 Casos criados pelo grupo

Entrada	Esperado	Obtido	
Próprio: [[3, 2, null], [4, null, 5], [null, 6, 1]]	sem_caminho	sem_caminho	**OK**
Próprio: {"custos": "1 2 3"}	entrada_invalida	entrada_invalida	**OK**
Próprio: [[1, 3, 1], [1, 5, 1], [4, 2, 1]]	ok, custo 7	ok, custo 7	**OK**
Próprio: [[1, 1, 1], [null, null, null], [1, 1, 1]]	sem_caminho	sem_caminho	**OK**
Próprio: [[1, 1, 1], [2, 9, 9], [1, 1, 1]]	ok, custo 6	ok, custo 6	**OK**
Próprio: [[null, 1, 1], [1, 1, 1]]	sem_caminho	sem_caminho	**OK**
Próprio: {"custos": null}	entrada_invalida	entrada_invalida	**OK**
Entrada	**Esperado**	**Obtido**	
Próprio: texto vazio	entrada_invalida	entrada_invalida	**OK**

3.3 Outros testes automáticos

- contagem de chamadas nas grades 2×3, 3×3, 6×6, 10×10 e 1×10 e na grade do exemplo (tarefa 5);
- regra gulosa: 26 contra 11 na grade do exemplo, grade própria 4×4 em que ela erra (23 contra 8), grade própria 3×3 em que ela erra (13 contra 6), grade própria em que ela fica presa, empate preferindo a direita, partida bloqueada e grade de uma casa (tarefa 6);
- recursivo não altera a grade recebida; saída não tem o campo caminho; entrada que não é texto é recusada.

3.4 Saída do executor de testes

```
test_empate_prefere_a_direita (test_robo.TestGuloso.test_empate_prefere_a_direita) ... ok test_grade_de_uma_casa
(test_robo.TestGuloso.test_grade_de_uma_casa) ... ok
test_grade_do_exemplo_da_26_contra_11 (test_robo.TestGuloso.test_grade_do_exemplo_da_26_contra_11) ... ok
test_grade_propria_4x4_em_que_o_guloso_erra (test_robo.TestGuloso.test_grade_propria_4x4_em_que_o_guloso_erra) ... ok
test_grade_propria_em_que_o_guloso_erra (test_robo.TestGuloso.test_grade_propria_em_que_o_guloso_erra) ... ok
test_grade_propria_em_que_o_guloso_fica_preso (test_robo.TestGuloso.test_grade_propria_em_que_o_guloso_fica_preso) ... ok test_partida_bloqueada
(test_robo.TestGuloso.test_partida_bloqueada) ... ok
test_chamadas_na_grade_do_exemplo (test_robo.TestMedicao.test_chamadas_na_grade_do_exemplo) ... ok
test_chamadas_nas_grades_de_uns (test_robo.TestMedicao.test_chamadas_nas_grades_de_uns) ... ok
test_grade_do_exemplo (test_robo.TestRecursivo.test_grade_do_exemplo) ... ok test_nao_altera_a_grade
(test_robo.TestRecursivo.test_nao_altera_a_grade) ... ok test_sem_rota_devolve_none
(test_robo.TestRecursivo.test_sem_rota_devolve_none) ... ok test_casos_do_grupo
(test_robo.TestResolver.test_casos_do_grupo) ... ok test_casos_do_professor
(test_robo.TestResolver.test_casos_do_professor) ... ok test_entrada_que_nao_e_texto
(test_robo.TestResolver.test_entrada_que_nao_e_texto) ... ok test_saida_sem_campo_caminho
(test_robo.TestResolver.test_saida_sem_campo_caminho) ... ok

----------------------------------------------------------------------
Ran 16 tests in 0.049s

OK
```

4. Por que funciona

Por que a recursão sempre termina?

De uma casa (linha, coluna) até a chegada faltam exatamente (última linha − linha) + (última coluna − coluna) movimentos. Cada chamada recursiva anda uma casa para a direita ou para baixo, então a cada chamada falta um movimento a menos. Esse número nunca fica negativo, porque só chamamos vizinhos que estão dentro da grade. Logo, depois de no máximo linhas + colunas − 2 chamadas encadeadas, chega-se à chegada (caso base) ou a um obstáculo (outro caso base), e a recursão para. Na pior das hipóteses a profundidade é linhas + colunas − 1.

Por que ela encontra o menor custo?

Toda rota que sai de uma casa que não é a chegada começa indo para a direita ou para baixo — não há outra opção. Então o menor custo a partir dela é o custo da casa mais o menor entre: o melhor custo a partir da casa da direita e o melhor custo a partir da casa de baixo. Supondo que as chamadas para os vizinhos (que estão mais perto da chegada) já devolvem o menor custo correto, escolher o menor dos dois dá o menor custo correto para a casa atual. A recursão examina, de forma implícita, todas as rotas possíveis e fica com a mais barata.

O que acontece quando não há rota?

Obstáculos devolvem None, e posições fora da grade nunca são escolhidas. Uma casa só devolve um número quando pelo menos um dos vizinhos devolveu um número; se os dois devolveram None, ela também devolve None. Assim o None “sobe” até a partida exatamente quando nenhuma rota consegue chegar.

5. Medição do trabalho

Usamos contar_chamadas(mapa), que executa exatamente a mesma recursão e conta quantas vezes a função foi chamada (incluindo a primeira chamada, na partida). Como o programa só chama vizinhos que estão dentro da grade, os números batem com a tabela do enunciado:

Grade (todas as casas = 1)	Esperado (enunciado)	Medido	C(L+C, L) − 1	Confere?
2 × 3	9	9	9	sim
3 × 3	19	19	19	sim
6 × 6	923	923	923	sim
10 × 10	184.755	184.755	184.755	sim
1 × 10	10	10	10	sim
Grade do exemplo (4 × 5)	—	43	125 sem obstáculos	—

Explicação dos números

Sem memorização, a função é chamada em uma casa uma vez para cada rota diferente que leva da partida até ela. Até a casa (i, j) existem C(i + j, i) rotas (escolher quais dos i + j passos são para baixo). Somando em todas as casas de uma grade L × C, o total de chamadas é **C(L + C, L) − 1**, coluna que calculamos acima e que coincide com a medição.

- **1 × 10**: só existe uma rota, então cada casa é chamada uma única vez: 10 chamadas.
- **2 × 3**: as casas recebem 1, 1, 1 / 1, 2, 3 chamadas = 9.

- **3 × 3:** com o aumento do tabuleiro, existem mais caminhos possíveis. A quantidade de chamadas aumenta para 19.
- **6 × 6**: o número de possibilidades cresce bastante, chegando a 923 chamadas.
- **10 × 10**: 184.755 chamadas para uma grade de apenas 100 casas. Só a casa de chegada é chamada C(18, 9) = 48.620 vezes. O trabalho cresce de forma exponencial, porque o mesmo pedaço da grade é recalculado muitas vezes.

Grade do exemplo

Na grade do exemplo foram medidas 43 chamadas (custo 11). Sem obstáculos, uma grade 4 × 5 teria 125. A diferença acontece porque os dois obstáculos também recebem chamadas, mas param imediatamente (caso base) e não geram chamadas para a frente; todas as rotas que passariam por eles são cortadas. A soma é 43, igual à medição, e as 7 chamadas na chegada correspondem às 7 rotas que existem nessa grade.

2	1	9	9	9
3	X	1	1	9
1	1	1	X	2
9	9	1	1	1

Custos

1	1	1	1	1
1	2	1	2	3
1	1	2	4	3
1	2	4	4	7

Chamadas por casa

6. Regra gulosa

guloso(mapa) começa na partida e, a cada passo, olha apenas os vizinhos **livres** (direita e baixo, dentro da grade e sem obstáculo) e anda para o **mais barato**. No **empate prefere a direita** (comparação direita <= baixo). Se não houver vizinho livre antes da chegada, o robô fica preso e a função devolve None. Ela nunca volta atrás nem olha o futuro — por isso pode errar.

6.1 Grade do exemplo: 26 contra 11

Resultado medido: **guloso = 26**, **recursivo = 11**.

2	1	9	9	9
3	X	1	1	9
1	1	1	X	2
9	9	1	1	1

Rota gulosa: 2+1+9+1+1+9+2+1 = 26

2	1	9	9	9
3	X	1	1	9
1	1	1	X	2
9	9	1	1	1

Rota ótima: 2+3+1+1+1+1+1+1 = 11

Onde a escolha atrapalhou: logo no primeiro passo o guloso compara 1 (direita) com 3 (baixo) e vai para a direita. Parece mais barato, mas leva à linha de cima, onde a casa de baixo do 1 é obstáculo e a única saída é um 9. Depois, na casa (1, 2), há empate entre 1 (direita) e 1 (baixo); a preferência pela direita leva a outro 9. A rota ótima aceita pagar 3 no início para descer pela primeira coluna e seguir só por casas de custo 1.

6.2 Grade própria (4 × 4): 23 contra 8

```
Entrada: {"custos": [[1, 1, 1, 1], [2, 9, 9, 9], [1, 9, 9, 9], [1, 1, 1, 1]]}
```

Resultado medido: **guloso = 23**, **recursivo = 8**.

1	1	1	1
2	9	9	9
1	9	9	9
1	1	1	1

Rota gulosa: 1+1+1+1+9+9+1 = 23

1	1	1	1
2	9	9	9
1	9	9	9
1	1	1	1

Rota ótima: 1+2+1+1+1+1+1 = 8

Onde a escolha atrapalhou: na partida o guloso compara 1 (direita) com 2 (baixo) e vai para a direita para “economizar” 1. Depois, em cada casa da primeira linha, a casa da direita (1) é sempre mais barata que a de baixo (9), então ele segue até o canto superior direito. Dali a única saída é descer pela última coluna, pagando 9 + 9 + 1. A rota ótima aceita pagar 2 no começo, desce pela primeira coluna e segue pela última linha só com casas de custo 1. Economizar 1 no primeiro passo custou 15 a mais no total (23 contra 8).

7. Conferência com os critérios de avaliação

Critério (pontos)	Onde está atendido
Descrição do problema (0,5)	Seção “Descrição do problema”: entrada, saída, objetivo, regras e regras de validade; exemplo próprio de entrada válida sem rota e de entrada inválida, com a saída real do programa.
Recursão e testes (1,0)	Seções “Solução recursiva” e “Testes”: recursivo sem memoização; 21 casos do professor e 8 casos do grupo; 16 testes automáticos, todos passando.
Por que funciona (0,6)	Seção “Por que funciona”: por que termina (falta um movimento a menos), por que acerta (toda rota começa pela direita ou por baixo) e como detecta a falta de rota.
Medição do trabalho (0,6)	Seção “Medição do trabalho”: chamadas medidas 9, 19, 923, 184.755 e 10 (iguais ao enunciado) e 43 na grade do exemplo, com fórmula e explicação por casa.
Regra gulosa (0,3)	Seção “Regra gulosa”: guloso com empate para a direita; 26 contra 11 na grade do exemplo; grade própria 4 × 4 com custos (23 contra 8) e explicação de onde a escolha atrapalhou.

Fora do escopo desta entrega (de propósito): memoização, tabela de resultados e o campo caminho com a lista de coordenadas, que ficam para a Entrega 2.

Como executar e arquivos entregues

Arquivo	Conteúdo
projeto_robo/robo.py	resolver, recursivo, guloso, contar_chamadas, validar_custos
tests/test_robo.py	testes automáticos (unittest)
main.py	demonstração pela linha de comando
README.md	instruções e explicação do código para a apresentação